



JOIG2021/2022 本選

A - ピアノコンクール

(Piano Competition)

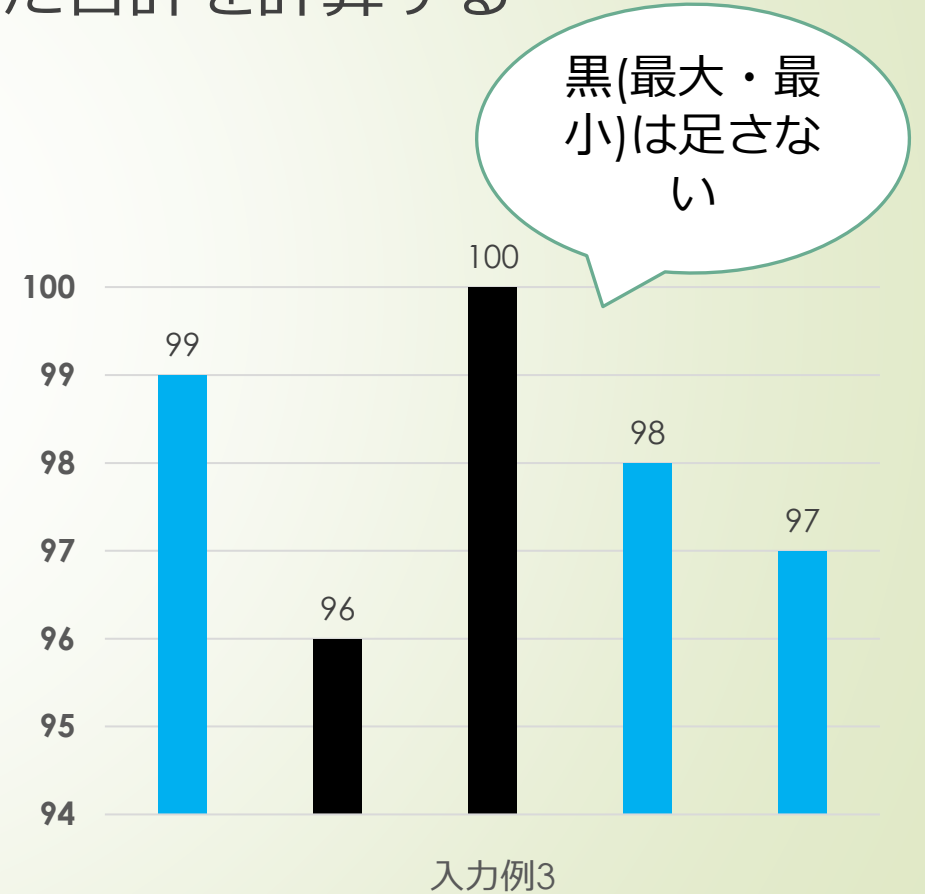
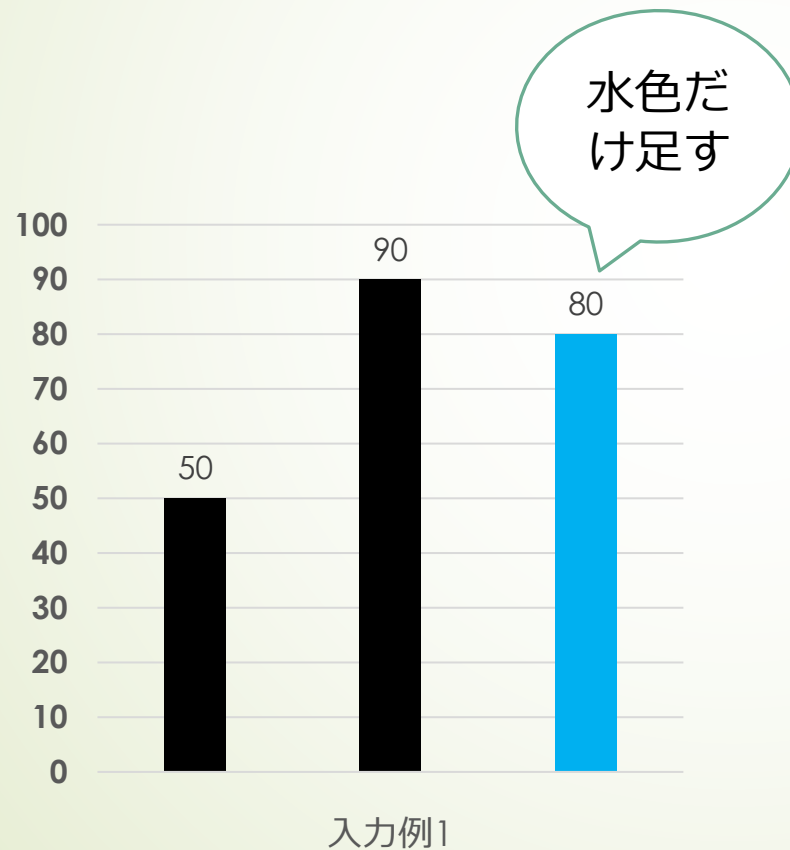
解説：米内山 匠実

問題概要

- N 人の審査員がそれぞれ点数をつける(互いに異なる)
- そのうち最高点と最低点の審査員を除いた合計が、「総合得点」
- 各審査員の点数が分かっている時、総合得点を計算する

言い換え

- ➡ N 個の整数がある(互いに異なる)
- ➡ そのうち最大値と最小値を除いた合計を計算する





言い換え

■ 最大値と最小値"以外"

→「以外」ということは.....

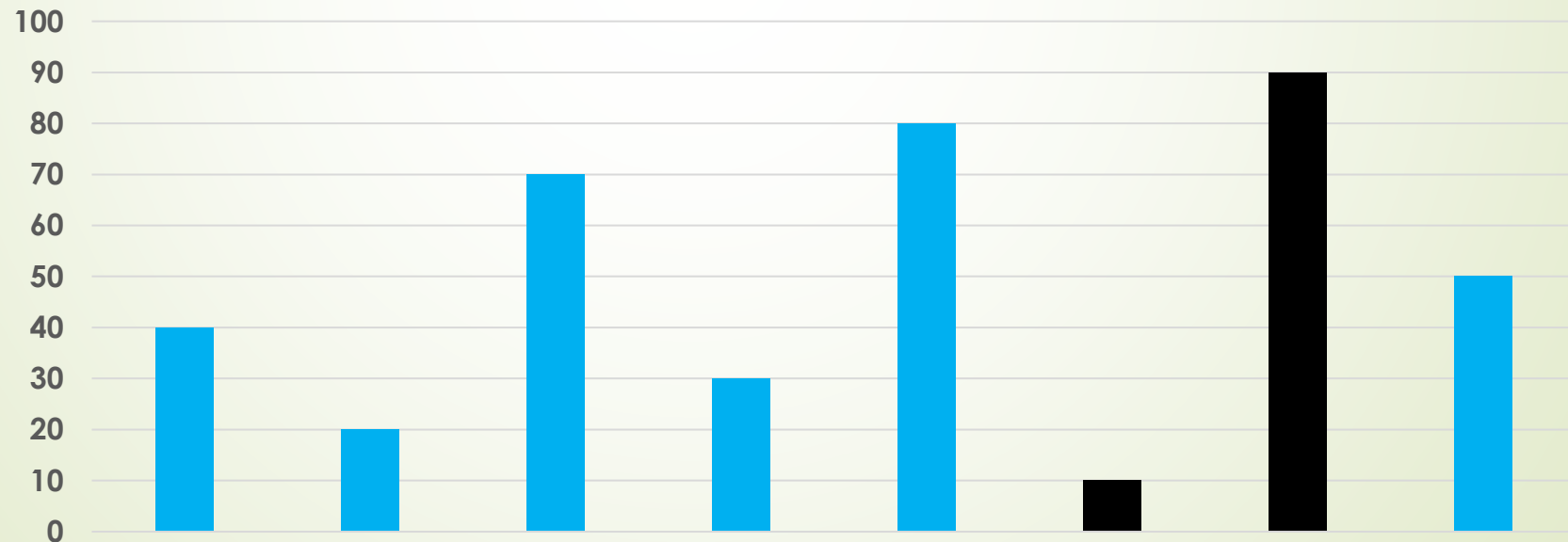
■ 最大値と最小値も含めた全ての合計から、最大値と最小値を引けばいい

言い換え

- ➡ 最大値と最小値も含めた全ての合計から、最大値と最小値を引けばいい

下の図で言えば.....

- ➡ 水色も黒も含めた全ての合計から、黒(最大と最小)を引く





言い換え

➡ つまり、求めるべきものは.....

- ① 全ての合計(最大も最小も含む)
- ② 最大値と最小値

最後に①から②を引けばいい

➡ どうやって実装する ???

実装

■ $N = 3$ の場合.....

■ A_1, A_2, A_3 が入力される。

① 全ての合計(最大も最小も含む)..... $A_1 + A_2 + A_3$

② 最大値と最小値..... C++ではmax関数、min関数を使える

※ $\max(\{A_1, A_2, A_3\})$ 、 $\min(\{A_1, A_2, A_3\})$ のように書ける

C++での実装例 ($N = 3$)

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int N, A1, A2, A3;
    cin >> N >> A1 >> A2 >> A3;
    cout << A1 + A2 + A3 - max({A1, A2, A3}) - min({A1, A2, A3}) << endl;
}
```

入力の受け
取り

A1, A2, A3
の最大値を
求める関数

A1, A2, A3
の最小値を
求める関数

実装(満点解法)

■ もっと大きい N の場合..... for文が使える！

① 全ての合計

$A_1, A_2, \dots$ と前から順番に入力を受け取る

例えば、 A_4 までの合計は、 A_3 までの合計と A_4 の和

→同様に考えて.....

それまでに受け取った数の合計を記録する変数を作る

新たに受け取った数を、その変数に足す

実装(満点解法)

■ もっと大きい N の場合..... for文が使える！

② 最大値と最小値

$A_1, A_2, \dots$ と順番に入力を受け取る

例えば、 A_4 までの最小値は、 A_3 までの最小値と A_4 の小さい方

→同様に考えて.....



実装(満点解法)

- ・ 最小値

それまでに受け取った数のうちの最小値を記録する変数を作る
新たに受け取った数と最小値を比べ、小さい方を新たな最小値にする

- ・ 最大値

それまでに受け取った数のうちの最大値を記録する変数を作る
新たに受け取った数と最大値を比べ、小さい方を新たな最大値にする

実装(満点解法)

■ 実装上の注意

- ・ 値は相異なるので、最小値の比較でイコールを入れるか、などは考えなくて良い

- ・ 和を記録する変数の初期値は、0にする

- ・ 最大値を記録する変数の初期値は、入ってくる値以下にする

----- 今回の制約は $0 \leq A_i \leq 100$ なので、初期値を0にすれば良い

----- 最小値の初期値も同様に、100にしておけば良い

- ・ 計算量は $O(N)$ となる

C++での実装例 (満点)

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int N, A, sum = 0, maximum = 0, minimum = 100;
    cin >> N;
    for(int i = 0; i < N; i++){
        cin >> A;
        sum += A;
        if(A > maximum){
            maximum = A;
        }
        if(A < minimum){
            minimum = A;
        }
    }
    cout << sum - maximum - minimum << endl;
}
```

C++での実装例 (満点)

```
#include <iostream>
#include <algorithm>
using namespace std;
int main() {
    int N, A, sum = 0, maximum = 0, minimum = 100;
    cin >> N;
    for(int i = 0; i < N; i++){
        cin >> A;
        sum += A;
        if(A > maximum){
            maximum = A;
        }
        if(A < minimum){
            minimum = A;
        }
    }
    cout << sum - maximum - minimum << endl;
}
```

合計を記録
する変数

最大値を記
録する変数

最小値を記
録する変数

合計値を更新

最大値を更新

最小値を更新



余談

- N 個の数の和、最大値などを求めるテクニックは頻出！
 - 「最大値を取る場所」も書けるようになろう
- 言語によっては簡単に求められることも
 - Pythonならsum, min, maxとかを使える
 - C++にも一応専用の方法がある

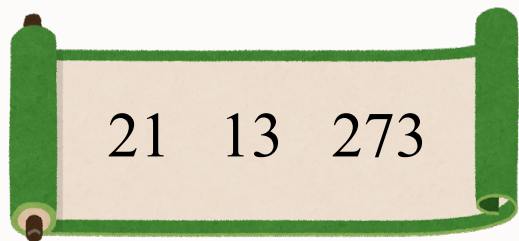
The background is a light beige color with several decorative elements: a blue circle with a dashed pattern in the top left, a pink circle in the top center, a large purple 'X' in a white circle in the top right, a wavy orange pattern in the far top right, a green and brown rolled-up scroll in the bottom left, an orange plus sign in the bottom center, and a blue plus sign in the bottom right.

JOIG 2021/2022本選 問題2 掛け算(Multiplication)

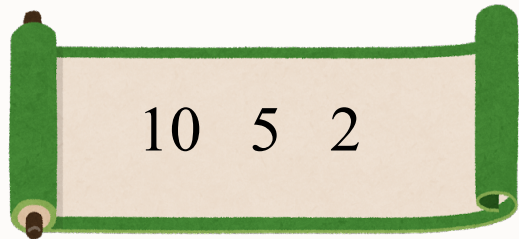
解説：@TumoiYorozu

問題概要

- 巻物に整数が N 個書かれています。
- この中から順番を変えずに3つ選び、その整数 x, y, z が $x \times y = z$ となる選び方は何通りありますか？

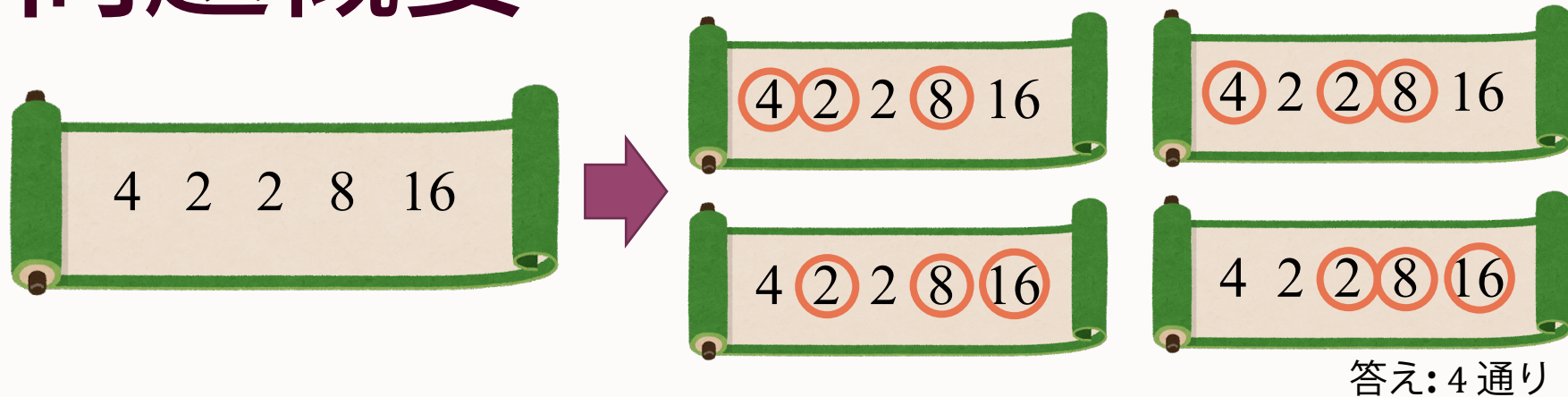


$21 \times 13 = 273$ であるので、
条件を満たす選び方は **(21, 13, 273)** の1通りのみ



$2 \times 5 = 10$ だが並び替えてはいけないので、
条件を満たす選び方は無い。答えは0通り

問題概要



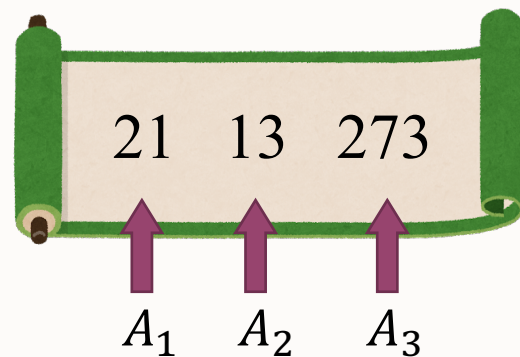
制約

巻物に書かれている整数の個数 N が、

- $N = 3$ のとき、**20**点
- $3 \leq N \leq 100$ のとき、更に **80** 点（満点）

考察 (小課題1: $N=3$ のとき)

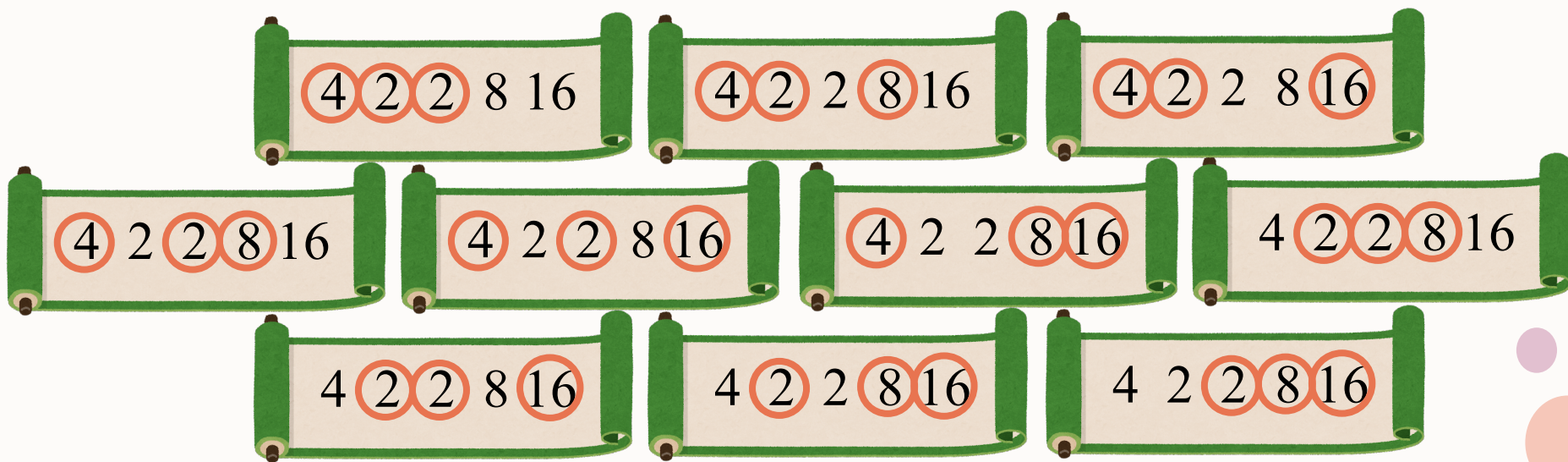
- 整数が3つしか無いので、考えるべき組み合わせは1通り
- この3つの整数 A_1, A_2, A_3 が、
 - $A_1 \times A_2 = A_3$ ならば、答えは1
 - $A_1 \times A_2 \neq A_3$ ならば、答えは0
- **if** などを使った条件分岐を用いることで得点することができる



考察 (満点: $N \leq 100$ のとき)

順番を変えずに3つ選び、その整数 x, y, z が $x \times y = z$ となる選び方は何通りありますか？

- 3つの選び方をすべて試して、条件に合うものをカウントする (全探索)
- 100個から3個を選ぶ組み合わせは**161,700**個あるが、コンピューターを使えば一瞬で調べることができる



全探索の仕方の考察

- 「全探索」と言っても、問題ごとにどう探索すべきかは異なる
- 手で全探索をしようとしたときを考えると、
 - ① まず1つめの数 x を決めて、
 - ② 次に x より右の中から、2つめの数 y を決めて、
 - ③ 次に y より右の中から、3つめの数 z を決めて、
 - ④ 最後に、この選んだ3つの数 x, y, z が、条件を満たすかをチェックを、順番に行う方法が考えられる



図: ①②で、 $4, 2$ の2つを決めたあと、③で3つめの数を順番に選ぶ様子

プログラムでこれをするには？

➤ 繰り返し文を入れ子にした、多重ループで行う

C++ での例

```
#include <iostream>
using namespace std;

int main() {
    int N;
    cin >> N;
    int A[100];
    for (int i = 0; i < N; ++i) {
        cin >> A[i];
    }

    int answer = 0;
    for (int i = 0; i < N; ++i) {
        for (int j = i + 1; j < N; ++j) {
            for (int k = j + 1; k < N; ++k) {
                if (A[i] * A[j] == A[k]) {
                    answer += 1;
                }
            }
        }
    }

    cout << answer << endl;
}
```

Python3での例

```
N = int(input())
A = list(map(int, input().split()))

answer = 0
for i in range(N):
    for j in range(i+1, N):
        for k in range(j+1, N):
            if A[i] * A[j] == A[k]:
                answer += 1

print(answer)
```

多重ループの解説

```
for (int i = 0; i < N; ++i) {  
    for (int j = i + 1; j < N; ++j) {  
        for (int k = j + 1; k < N; ++k) {  
            if (A[i] * A[j] == A[k]) {  
                answer += 1;  
            }  
        }  
    }  
}
```

C++ での例

- まず、最初の数 x を選ぶ
- i は何番目の数を選んだかを表す。 $x = A_i$
- i を 1 番目～ N 番目の中で順番に試すので **for** を使って繰り返す
- プログラミングでは**0**番目から数えるので、 i を**0**から始めて、 N 未満まで試す

多重ループの解説

```
for (int i = 0; i < N; ++i) {  
    for (int j = i + 1; j < N; ++j) {  
        for (int k = j + 1; k < N; ++k) {  
            if (A[i] * A[j] == A[k]) {  
                answer += 1;  
            }  
        }  
    }  
}
```

C++ での例

- 次に、2 番目の数 y を選ぶ
- j は何番目の数を選んだかを表す。 $y = A_j$
- y は、1 番目の数 x よりも後ろなので、 j は $i + 1$ から N 番目の中で順番に試す

多重ループの解説

```
for (int i = 0; i < N; ++i) {  
    for (int j = i + 1; j < N; ++j) {  
        for (int k = j + 1; k < N; ++k) {  
            if (A[i] * A[j] == A[k]) {  
                answer += 1;  
            }  
        }  
    }  
}
```

C++ での例

- 最後に、3番目の数 z を選ぶ
- k は何番目の数を選んだかを表す。 $z = A_k$
- z は、2番目の数 y よりも後ろなので、 k は $j + 1$ から N 番目の中で順番に試す

多重ループの解説

```
for (int i = 0; i < N; ++i) {  
    for (int j = i + 1; j < N; ++j) {  
        for (int k = j + 1; k < N; ++k) {  
            if (A[i] * A[j] == A[k]) {  
                answer += 1;  
            }  
        }  
    }  
}
```

C++ での例

- i, j, k が選ばれたので、条件を満たすか **if** 文で判定し、条件に合うならば **answer** に 1 を足す

まとめ

- コンピューターを使えば、手計算では量が多くて時間がかかる事を、高速に処理してくれる
- 複雑な処理でも、**for** や **if** などを組み合わせ、ときにはたくさん入れ子にすることで、求めることができる

JOIG 2021/2022 本選 3問目

Voting 解説

大佐 健人

問題概要

- N 人が順番に「賛成」 / 「反対」に投票する
- 直前に投票した X_i 人のうち Y_i 人以上が賛成 → 賛成に投票
- そうでない → 反対に投票
- 賛成に投票したのは何人？



小課題 1 ($N \leq 3000$)

- 問題文通りにシミュレーション
- for 文で $1, 2, \dots, N$ 番目の人を順に見ていく
- i 番目の人について、どちらに投票するか判定したい

小課題 1 ($N \leq 3000$)

- 配列 A に、それぞれの人の投票先を保存しておく
 - 賛成なら $A[i] = 1$, 反対なら $A[i] = 0$ など
- for 文で、 $A[i - 1], A[i - 2], \dots, A[i - X_i]$ の中の賛成の個数を数える
 - 計算量は $O(N)$
- 賛成が Y_i 人以上なら賛成に投票
 - $A[i]$ を更新

小課題 1 ($N \leq 3000$)

- 全員の投票先が分かるので、解けた
- このような処理は二重の for 文でできる
- 計算量は $O(N^2)$
 - i を 1 から N まで動かし、各 i について判定に $O(N)$ かかるため
 - この計算量だと満点は間に合わない...
- 28点

小課題 1 ($N \leq 3000$)

- 実装例 (C++)

```
int N;
cin >> N;
vector<int> X(N), Y(N), A(N);
for (int i = 0; i < N; i++) {
    cin >> X[i] >> Y[i];
}
int res = 0;
for (int i = 0; i < N; i++) {
    int cnt = 0;
    for (int j = i - X[i]; j < i; j++) {
        cnt += A[j];
    }
    if (cnt >= Y[i]) {
        //賛成に投票
        res++;
        A[i] = 1;
    }
}
cout << res << endl;
```

小課題 2 ($X_i = i - 1$)

- Q. この小課題の意味は？
- A. それぞれの人について、今までに投票した**全員**の中で何人賛成したかが分かれば、投票先が分かる
- 「**今までに賛成した人数**」を変数として持てばよい！
 - これを変数 c とする

小課題 2 ($X_i = i - 1$)

- 条件： $Y_i \leq c$ なら賛成、そうでないなら反対
- 賛成なら c を 1 増やす
- これを N 人について、順番に行えば解ける
- 32点

小課題 2 ($X_i = i - 1$)

- 実装例 (C++)

```
int N;
cin >> N;
vector<int> X(N), Y(N);
for (int i = 0; i < N; i++) {
    cin >> X[i] >> Y[i];
}
int c = 0, res = 0;
for (int i = 0; i < N; i++) {
    if (Y[i] <= c) {
        //賛成に投票
        c++;
        res++;
    }
}
cout << res << endl;
```

満点解法

- $N \leq 500000 \rightarrow$ 小課題 1 の $O(N^2)$ は間に合わない
- それぞれの人について、投票先を高速に決めたい
- 小課題 2 のアイデアを応用できないか…？

満点解法

- 小課題 2 では $X_i = i - 1$ だったから、「 $i - 1$ 番目までに何人賛成したか」を使えば判定できた
- 重要な事実：
(直前 X_i 人で賛成した人数)
 $= (i - 1 \text{ 番目までに賛成した人数}) - (i - X_i - 1 \text{ 番目までに賛成した人数})$

満点解法

○	×	○	○	○	×	×	○
---	---	---	---	---	---	---	---



直前 5 人のうち賛成した人数 = 3

||



8番目までに賛成した人数 = 5

|



3番目までに賛成した人数 = 2

(○ : 賛成、× : 反対)

満点解法

- $S[i]$: i 番目までに賛成した人数 という配列を作っておく
- "重要な事実" を書き直すと...
- (直前 X_i 人で賛成した人数) = $S[i - 1] - S[i - X_i - 1]$
- $O(1)$ で計算できた！

満点解法

- $S[i]$ 自体も、 $S[i - 1]$ が分かっているならば $O(1)$ で計算できる
 - 賛成なら $S[i] = S[i - 1] + 1$ 、反対なら $S[i] = S[i - 1]$
- このような高速化テクニックを「**累積和**」という
 - 重要！
- 計算量は全体で $O(N)$ →間に合う
- 100点

満点解法

- 実装例 (C++) (入力部が小課題1,2と違うことに注意)

```
int N;
cin >> N;
vector<int> X(N + 1), Y(N + 1), S(N + 1);
for (int i = 1; i <= N; i++) {
    cin >> X[i] >> Y[i];
}
int res = 0;
for (int i = 1; i <= N; i++) {
    S[i] = S[i - 1];
    if (S[i - 1] - S[i - 1 - X[i]] >= Y[i]) {
        res++;
        S[i]++;
    }
}
cout << res << endl;
```



いちご 2 (STRAWBERRY 2)

解説：平木 康傑

問題概要

- $H \times W$ のマス目に N 個のいちごがある
- 3×3 の範囲をとる
- とれるいちごは最大でいくつ？

問題概要

- $H \times W$ のマス目に N 個のいちごがある
 - 3×3 の範囲をとる
 - とれるいちごは最大でいくつ？
-
- 小課題 1 (14 点): $H, W \leq 1000, N \leq 20$
 - 小課題 2 (16 点): $H, W \leq 1000$
 - 小課題 3 (29 点): $H \leq 10^6, N \leq 500$
 - 小課題 4 (15 点): $H = 3$
 - 小課題 5 (20 点): $H \leq 10^6$
 - 小課題 6 (6 点): H も W も N も大きい

小課題 1 (14 点)

小課題 1 (14 点): $H, W \leq 1000, N \leq 20$
小課題 2 (16 点): $H, W \leq 1000$
小課題 3 (29 点): $H \leq 10^6, N \leq 500$
小課題 4 (15 点): $H = 3$
小課題 5 (20 点): $H \leq 10^6$
小課題 6 (6 点): H も W も N も大きい

- $H, W \leq 1000$ なので, とりえる 3×3 範囲を全部列挙できる
- 各範囲内のいちごの個数を数え, その最大値を取ればよい
- $N \leq 20$ なので, 各範囲に対してそれぞれのいちごが範囲内かを調べても十分高速
- 時間計算量は $\Theta(HWN)$ となり, 間に合う

小課題 2 (16 点)

小課題 1 (14 点): $H, W \leq 1000, N \leq 20$
小課題 2 (16 点): $H, W \leq 1000$
小課題 3 (29 点): $H \leq 10^6, N \leq 500$
小課題 4 (15 点): $H = 3$
小課題 5 (20 点): $H \leq 10^6$
小課題 6 (6 点): H も W も N も大きい

- $H, W \leq 1000$ なので, とりえる 3×3 範囲を全部列挙できる
- 各範囲内のいちごの個数を数え, その最大値を取ればよい
- マス目に対応する 2 次元配列をとり, 各マスにあるいちごの個数を記録する
- 各範囲について, 9 マスの和をとればよい
- 時間計算量は $\Theta(HW)$ となり, 間に合う
- 余談: 制約が小課題 1 の上位互換なので, こちらを解くだけで小課題 1 も解けたことになり, 計 30 点を得られる

小課題 3 (29 点)

小課題 1 (14 点): $H, W \leq 1000, N \leq 20$
小課題 2 (16 点): $H, W \leq 1000$
小課題 3 (29 点): $H \leq 10^6, N \leq 500$
小課題 4 (15 点): $H = 3$
小課題 5 (20 点): $H \leq 10^6$
小課題 6 (6 点): H も W も N も大きい

- とりえる 3×3 範囲がたくさん
 - 本当にたくさん？
- 各いちごについて, それを含む 3×3 範囲はちょうど 9 通り
- ということは, いちごを 1 個でも含むような範囲は高々 $9N$ 通り
- なので, いちごを 1 個でも含むような範囲のみを列挙すれば大幅に探索範囲を絞れる

小課題 3 (29 点)

小課題 1 (14 点): $H, W \leq 1000, N \leq 20$
小課題 2 (16 点): $H, W \leq 1000$
小課題 3 (29 点): $H \leq 10^6, N \leq 500$
小課題 4 (15 点): $H = 3$
小課題 5 (20 点): $H \leq 10^6$
小課題 6 (6 点): H も W も N も大きい

- とりえる 3×3 範囲がたくさん
 - 本当にたくさん？
- 各いちごについて、それを含む 3×3 範囲はちょうど 9 通り
- ということは、いちごを 1 個でも含むような範囲は高々 $9N$ 通り
- なので、いちごを 1 個でも含むような範囲のみを列挙すれば大幅に探索範囲を絞れる
- 実際には、被ってもいいのでとにかく各いちごについて周囲 9 範囲を調べればよい
 - 最大値が分かればいいので、同じ範囲を何回調べても支障はない
- これと小課題 1 の解法 (範囲それぞれについて、各いちごが範囲内かを見る) を組み合わせて、 $\Theta(N^2)$
- 下位互換として取れる小課題 1 と合わせて 43 点が得られる

小課題 4 (15 点)

小課題 1 (14 点): $H, W \leq 1000, N \leq 20$
小課題 2 (16 点): $H, W \leq 1000$
小課題 3 (29 点): $H \leq 10^6, N \leq 500$
小課題 4 (15 点): $H = 3$
小課題 5 (20 点): $H \leq 10^6$
小課題 6 (6 点): H も W も N も大きい

- $H = 3$ なので, 実質 1 次元の問題
 - その代わり, 横幅は広い
- 調べる範囲を $9N$ 通りに絞るまでは小課題 3 と同様
- 「あるマスにいちごがいくつあるか」(ある数がいくつあるか)を調べるには, 次のような方針がある
 - 配列に格納してソートし, 二分探索や尺取り法などを適用
 - 連想配列 (C++ なら `std::map`) に入れて, 通常の配列と同様に座標にアクセス
- 二分探索や連想配列なら $\Theta(N \log N)$, 尺取り法なら $\Theta(N)$ で解ける
- 時間制限がやや厳しいので `std::vector` ではなく生の配列を使う方が通りやすい
- ともかく 15 点

小課題 5, 6 (満点)

小課題 1 (14 点): $H, W \leq 1000, N \leq 20$
小課題 2 (16 点): $H, W \leq 1000$
小課題 3 (29 点): $H \leq 10^6, N \leq 500$
小課題 4 (15 点): $H = 3$
小課題 5 (20 点): $H \leq 10^6$
小課題 6 (6 点): H も W も N も大きい

- 小課題 4 の解法を 2 次元に拡張する
- 二分探索や連想配列といった方針の場合, 書き換えるのは簡単
 - 2 次元座標をそのまま添え字とした配列/`std::vector`/連想配列を使えばそのまま拡張できる
- 尺取り法の場合は少しややこしいが, 特別新しい発想は必要ない
 - 範囲をスライスして 3 分割するとラク
- 上は $\Theta(N \log N)$ のまま, 下は $\Theta(N)$ のままで, 十分高速に動く
- 満点
- おめでとう

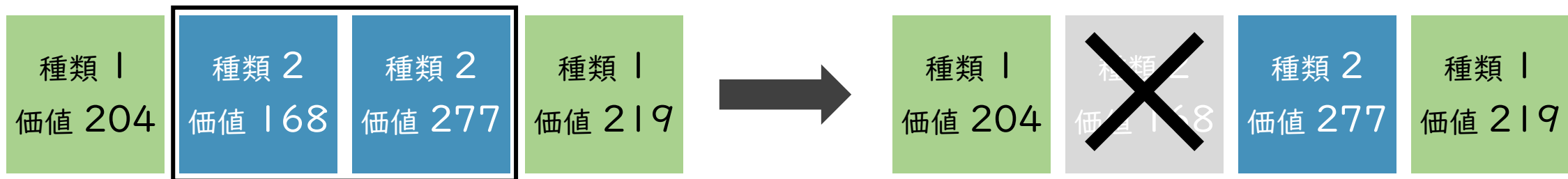
問題5：エゴイ展 解説

情報オリンピック女性部門 2022 本選

米田 優峻 (@e869120)

- 横一列に N 個の絵が並べられています。
- i 個目の絵の種類は A_i 、価値は V_i です。
- 隣り合う絵が同じ種類にならないように絵を撤去するとき、残った絵の価値合計の最大値はいくつですか？

$N = 4, (A_i, V_i) = (1, 204), (2, 168), (2, 277), (1, 219)$ の場合



隣り合う絵が同じ色なので
撤去しなければならない

このようにすれば条件を満たす
価値合計は $204 + 277 + 219 = 700$

小課題 Ⅰ

$$M = 1, V_i \geq 1 \ (1 \leq i \leq N)$$

小課題 1 の解法

5 / 49

- 制約が $M = 1$ なので、全部の絵の色が同じです。
- 絵を 1 つしか残すことができません。
- 価値が最大となる絵を残すのが最適なので、求めるべき答えは $\max(V_1, V_2, \dots, V_N)$ となります。

小課題 1 の実装

6 / 49

- このように、簡単に実装することができます。
- 情報オリンピックでは、取れる部分点は全部取りましょう。

```
# 入力
N = int(input())
M = int(input())
A = [None] * N
V = [None] * N
for i in range(N):
    A[i], V[i] = map(int, input().split());
# 出力
print(max(V))
```

4点

小課題 2

$$V_i \geq 1 \ (1 \leq i \leq N)$$

2 小課題 2 の解法

8 / 49

- まずは以下のケースでの答えを考えてみよう
- 1～3 番目の中からは、最大 1 つしか選べない
- 4～6 番目の中からは、最大 1 つしか選べない
- 7～9 番目の中からは、最大 1 つしか選べない



2 小課題 2 の解法

9 / 49

- まずは以下のケースでの答えを考えてみよう
- 1～3 番目の中からは、最大 1 つしか選べない
- 4～6 番目の中からは、最大 1 つしか選べない
- 7～9 番目の中からは、最大 1 つしか選べない

選べる最大のものを選ぼう！

→ 答え： $29+37+34=100$



2

小課題 2 の解法

10 / 49

- まずは以下のケースでの答えを考えてみよう

- 1~3 番目の中からは、最大 1 つしか選べない

- 4~6 番目の中からは、最大 1 つしか選べない
- 他のケースでも同じ方法で選べる最大のものを選ぼう！
→ 答え：29+37+34=100

- 7~9 番目の中からは、最大 1 つしか選べない

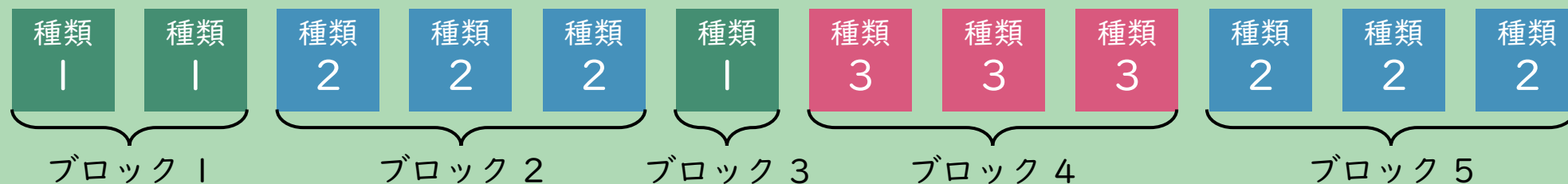
答えが求められる



2 小課題 2 の解法

以下のアルゴリズムにより、答えが求められる

- N 個の絵を「同じ種類からなるブロック」に分割する
- 各ブロックについて、価値の最大値 (*) を求める
- 求める答えは (*) の合計



2 小課題 2 の実装

12 / 49

```
# 入力
N = int(input())
M = int(input())
A = [None] * N
V = [None] * N
for i in range(N):
    A[i], V[i] = map(int, input().split());

# 答えを求める
Answer = 0
BlockMax = 0
for i in range(N):
    BlockMax = max(BlockMax, V[i])
    if (i == N-1) or (A[i] != A[i+1]):
        Answer += BlockMax
        BlockMax = 0

# 出力
print(Answer)
```

実装はやや大変だが、左のような実装例が考えられる

※Python での実装例

21 点

小課題 3

$$N \leq 15$$

3 全探索できるのか？

14 / 49

まず、絵の残し方は全部で何通りあるのかを考えよう

- 絵 1 の残し方： Yes/No の 2 通り
- 絵 2 の残し方： Yes/No の 2 通り
- 絵 3 の残し方： Yes/No の 2 通り
- :
- 絵 N の残し方： Yes/No の 2 通り

積の法則 (*)より、残し方は全部で

$$2 \times 2 \times \cdots \times 2 = 2^N \text{ 通り}$$

たとえば $N = 3$ のときは 8 通り

1	2	3
1	2	3
1	2	3
1	2	3
1	2	3
1	2	3
1	2	3
1	2	3

(*) 知らない人は、 <https://manabitimes.jp/math/1256> を確認しましょう！

3 全探索できるのか？

15 / 49

まず、絵の残し方は全部で何通りあるのかを考えよう

- 絵 1 の残し方： Yes/No の 2 通り

- 絵 2 の残し方： Yes/No の 2 通り

- 絵 3 の残し方： Yes/No の 2 通り

- 絵 N の残し方： Yes/No の 2 通り

小課題 3 の制約は $N \leq 15$

全部の残し方を調べることは可能か？

積の法則 (*) より、残し方は全部で

$$2 \times 2 \times \dots \times 2 = 2^N \text{ 通り}$$

たとえば $N=3$ のときは 8 通り

1	2	3
1	2	3
1	2	3
1	2	3
1	2	3
1	2	3
1	2	3
1	2	3

(*) 知らない人は、 <https://manabitimes.jp/math/1256> を確認しましょう！

3 全探索できるのか？

16 / 49

- $N = 15$ の場合、全部で $2^{15} = 32768$ 通りを調べる必要がある
- 一方、コンピュータの計算速度は 1 秒当たり 10 億回程度
- したがって、全探索をすると小課題 3 が解ける



問題：どうやって実装するか？

3 単純に実装すると...

17 / 49

// C++ での実装

```
for (int p1 = 0; p1 <= 1; p1++) {  
    for (int p2 = 0; p2 <= 1; p2++) {  
        for (int p3 = 0; p3 <= 1; p3++) {  
            for (int p4 = 0; p4 <= 1; p4++) {  
                for (int p5 = 0; p5 <= 1; p5++) {  
                    for (int p6 = 0; p6 <= 1; p6++) {  
                        :  
                    }  
                }  
            }  
        }  
    }  
}
```

- 絵 i を残したかどうかを P_i とする
- ここで P_i は 0 または 1
- 単純に for 文ループを書いて全探索することも可能だが、 N 重ループとなり実装が大変

3 実装の工夫：ビット全探索

18 / 49

- この問題を解消するため、**ビット全探索**という方法が用いられる
- for 文で $0 \leq t \leq 2^N - 1$ の範囲を全探索し、 t の下から i 桁目が 1 のとき絵 i を選ぶとする手法（以下は $N = 3$ の例）

i の値	0	1	2	3	4	5	6	7
絵 3 を選ぶか P_3	0	0	0	0	1	1	1	1
絵 2 を選ぶか P_2	0	0	1	1	0	0	1	1
絵 1 を選ぶか P_1	0	1	0	1	0	1	0	1

3 参考資料

19 / 49

- ビット全探索に関する詳しい説明
 - 「アルゴリズム×数学」 p.66
 - <https://drken1215.hatenablog.com/entry/2019/12/14/171657>
- 10 進数を 2 進数に変換する方法
 - 「アルゴリズム×数学」 p.21
 - <https://webkaru.net/clang/decimal-to-binary/>

42点

小課題 4

$$N \leq 100$$

4 小課題 4 の解法

21 / 49

- 小課題 3 と同様に全探索をすると、大変なことになる
- 全部で $2^{100} = 1267650600228229401496703205376$ 通りを探索しなければならない
- コンピュータの計算速度は 1 秒に 10 億回程度であるため、単純計算すると計算に 40 兆年かかる

4 小課題 4 の解法

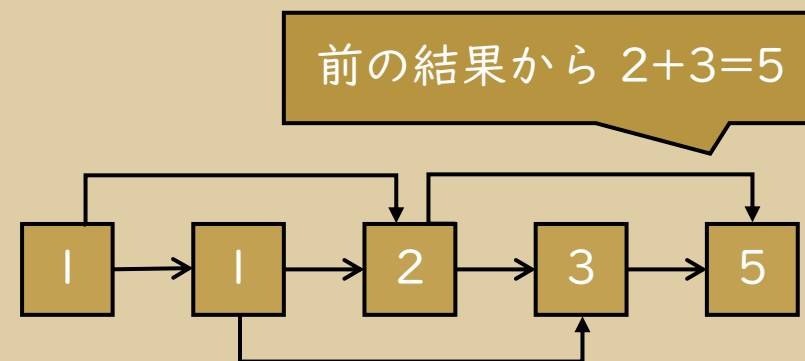
22 / 49

- ではどうするか？ → 以下の動的計画法を使おう！

動的計画法（DP）とは

前の結果を利用して問題を解く設計技法。フィボナッチ数などがイメージしやすい。詳しくは

<https://qiita.com/drken/items/dc53c683d6de8aeacf5a>



4 小課題 4 の解法

23 / 49

- この問題で動的計画法を適用させよう
- $dp[x] :=$ (絵 $1, 2, \dots, x$ まで考えたとき、絵 x を取った場合の価値合計の最大値) とする
- 絵 i の隣に絵 x を置ける条件は $A_i \neq A_x$ であるため、 $dp[x]$ は $A_i \neq A_x$ を満たすような i における $dp[i] + V_i$ の最大値である

➡ 難しいので、例を考えてみよう！

4 具体例

24 / 49

- たとえば、以下のようなケースを考える



4 具体例

25 / 49

- たとえば、以下のようなケースを考える
 - $dp[1]$ は「絵 1 を残すときの合計価値の最大値」
 - 絵 1 を取るしか選択肢がないので、 $dp[1] = dp[0] + V_1 = 7$



4 具体例

26 / 49

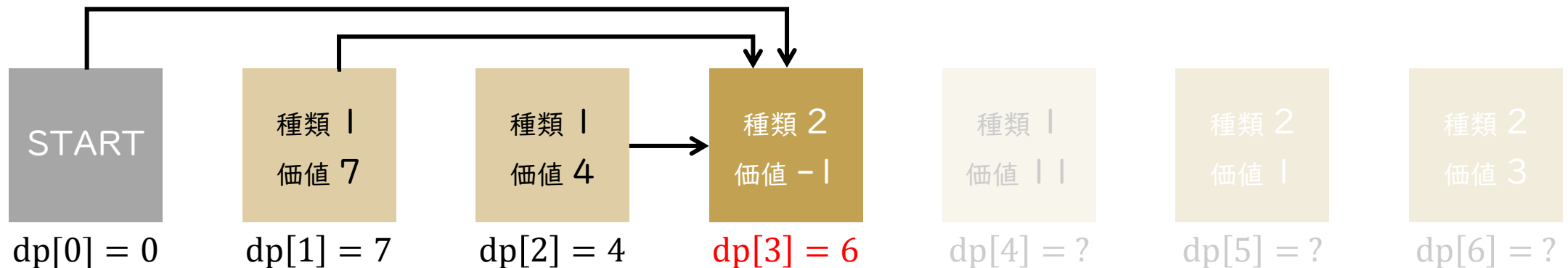
- たとえば、以下のようなケースを考える
 - $dp[2]$ は「絵 1 を残すときの合計価値の最大値」
 - 絵 2 は絵 1 の隣に置くことができないので、 $dp[2] = dp[0] + V_2 = 4$



4 具体例

27 / 49

- たとえば、以下のようなケースを考える
 - $dp[2]$ は「絵 1 を残すときの合計価値の最大値」
 - 絵 3 は一番左に置くか、絵 1・2 の隣に置くことができる
 - よって、 $dp[3] = \max(dp[0], dp[1], dp[2]) + V_3 = 6$



4 具体例

28 / 49

- たとえば、以下のようなケースを考える
 - $dp[2]$ は「絵 1 を残すときの合計価値の最大値」
 - 絵 4 は一番左に置くか、絵 3 の隣に置くことができる
 - よって、 $dp[4] = \max(dp[0], dp[3]) + V_4 = 17$



4 具体例

29 / 49

- たとえば、以下のようなケースを考える
 - $dp[2]$ は「絵 1 を残すときの合計価値の最大値」
 - 絵 5 は一番左に置くか、絵 1・2・4 の隣に置くことができる
 - よって、 $dp[5] = \max(dp[0], dp[1], dp[2], dp[4]) + V_5 = 18$



4 具体例

30 / 49

- たとえば、以下のようなケースを考える
 - $dp[2]$ は「絵 1 を残すときの合計価値の最大値」
 - 絵 6 は一番左に置くか、絵 1・2・4 の隣に置くことができる
 - よって、 $dp[6] = \max(dp[0], dp[1], dp[2], dp[4]) + V_6 = 20$



4 具体例

31 / 49

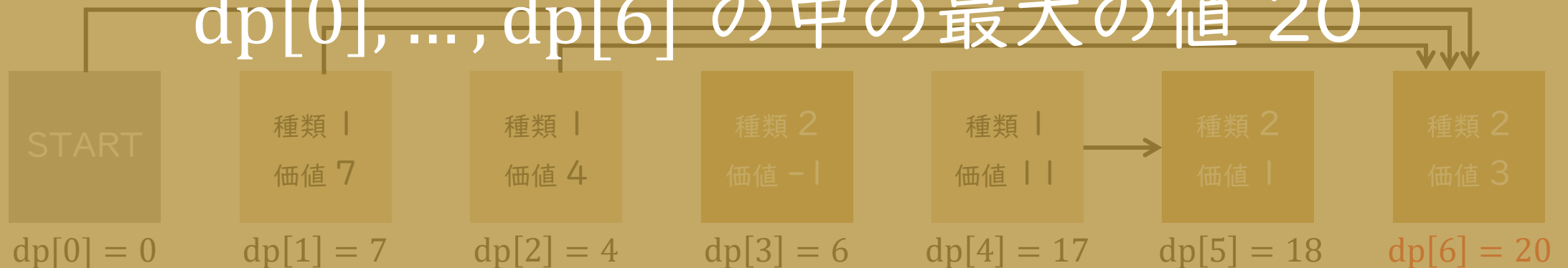
- たとえば、以下のようなケースを考える

- $dp[2]$ は「絵 1 を残すときの合計価値の最大値」

- 求める答え（残す絵の合計価値の最大）は

- よって、 $dp[6] = \max(dp[0], dp[1], dp[2], dp[4]) + V_6 = 20$

$dp[0], \dots, dp[6]$ の中の最大の値 20



4 小課題 4 の解法

32 / 49

```
// C++ での実装
// 動的計画法
dp[0] = 0;
for (int i = 1; i <= N; i++) {
    dp[i] = dp[0] + V[i];
    for (int j = 1; j < i; j++) {
        if (A[j] != A[i]) {
            dp[i] = max(dp[i], dp[j] + V[i]);
        }
    }
}

// 答え Ans を求める
int Ans = 0;
for (int i = 0; i <= N; i++) Ans = max(Ans, dp[i]);
cout << Ans << endl;
```

- 一般のケースでも、左のようなプログラムに従って計算すれば良い
- 計算量は $O(N^2)$ なので、 $N \leq 100$ ならば通る

69点

小課題 5

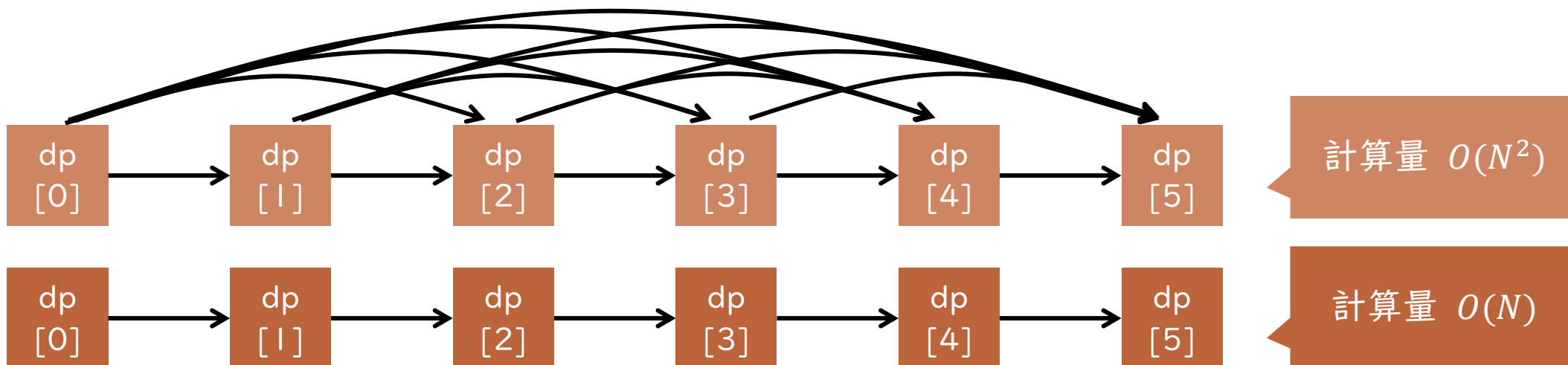
$$M \leq 100$$

5

計算量を減らす方法

34 / 49

- 小課題 4 の解法の計算が遅い理由は、一つの pos に対して $dp[pos]$ を計算するのに $O(N)$ 時間かかるから
- 上の状態遷移を、下の状態遷移のようにできないか？



5

小課題 5 の解法

35 / 49

- まずは $M = 2$ の場合について、以下の DP 配列を考えよう
 - $dp1[x]$: 絵 $1, \dots, x$ のみを考えたとき、最後に残した絵の種類が 1 の場合の合計価値最大値
 - $dp2[x]$: 絵 $1, \dots, x$ のみを考えたとき、最後に残した絵の種類が 2 の場合の合計価値最大値
- ※最後に残した絵: 絵 $1, 2, \dots, x$ の中で残したもののうち、最も番号が大きい絵 (一番右の絵) を指す。

- 同じ種類の絵を隣に置くことはできないので、答えは以下のように計算できる
 - $A_i = 1$ のとき: $dp1[i] = \max(\text{直前が種類 1 の絵で絵 } i \text{ を残さない}, \text{直前が種類 2 の絵で絵 } i \text{ を残す} + V_i)$, $dp2[i] = \text{直前が種類 2 の絵で絵 } i \text{ を残さない}$
 - $A_i = 2$ のとき: $dp2[i] = \max(\text{直前が種類 2 の絵で絵 } i \text{ を残さない}, \text{直前が種類 1 の絵で絵 } i \text{ を残す} + V_i)$, $dp1[i] = \text{直前が種類 1 の絵で絵 } i \text{ を残さない}$

5

小課題 5 の解法

36 / 49

- 難しいので、たとえば以下のようなケースを考えよう

dp1[0] 0	dp1[1] ?	dp1[2] ?	dp1[3] ?	dp1[4] ?	dp1[5] ?	dp1[5] ?
dp2[0] 0	dp2[1] ?	dp2[2] ?	dp2[3] ?	dp2[4] ?	dp2[5] ?	dp2[5] ?
	種類 1 / 価値 7	種類 1 / 価値 4	種類 2 / 価値 -1	種類 1 / 価値 11	種類 2 / 価値 1	種類 2 / 価値 3

5

小課題 5 の解法

37 / 49

- 難しいので、たとえば以下のようなケースを考えよう

- $dp1[1] = \max(dp1[0], dp2[0] + 7) = 7$ [最後に残した絵が種類 1 のときの合計価値最大]
- $dp2[1] = dp2[0] = 0$ [最後に残した絵が種類 2 のときの合計価値最大]



5

小課題 5 の解法

38 / 49

- 難しいので、たとえば以下のようなケースを考えよう

- $dp1[2] = \max(dp1[1], dp2[1] + 4) = 7$ [最後に残した絵が種類 1 のときの合計価値最大]
- $dp2[2] = dp2[1] = 0$ [最後に残した絵が種類 2 のときの合計価値最大]



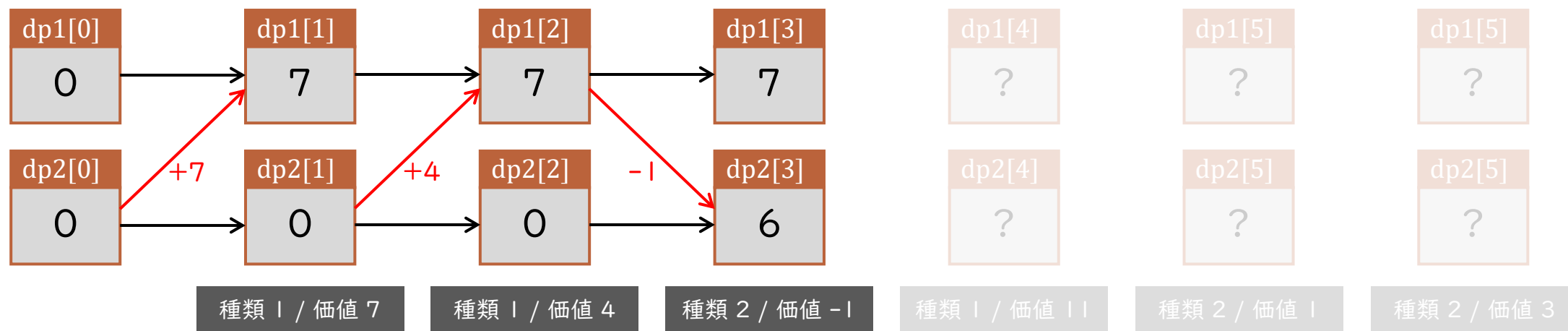
5

小課題 5 の解法

39 / 49

- 難しいので、たとえば以下のようなケースを考えよう

- $dp1[3] = dp1[2] = 7$ [最後に残した絵が種類 1 のときの合計価値最大]
- $dp2[3] = \max(dp2[2], dp1[2] - 1) = 6$ [最後に残した絵が種類 2 のときの合計価値最大]



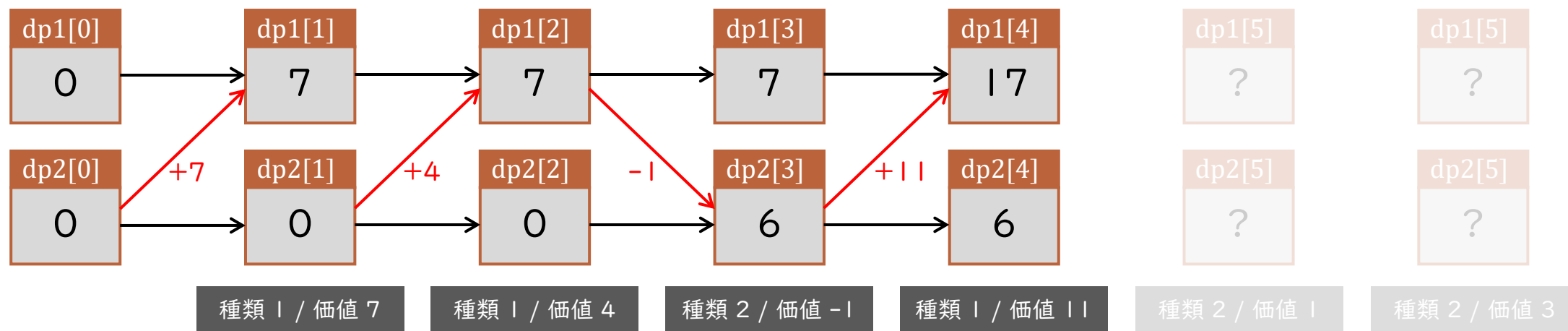
5

小課題 5 の解法

40 / 49

- 難しいので、たとえば以下のようなケースを考えよう

- $dp1[4] = \max(dp1[3], dp2[3] + 11) = 17$ [最後に残した絵が種類 1 のときの合計価値最大]
- $dp2[4] = dp2[3] = 6$ [最後に残した絵が種類 2 のときの合計価値最大]



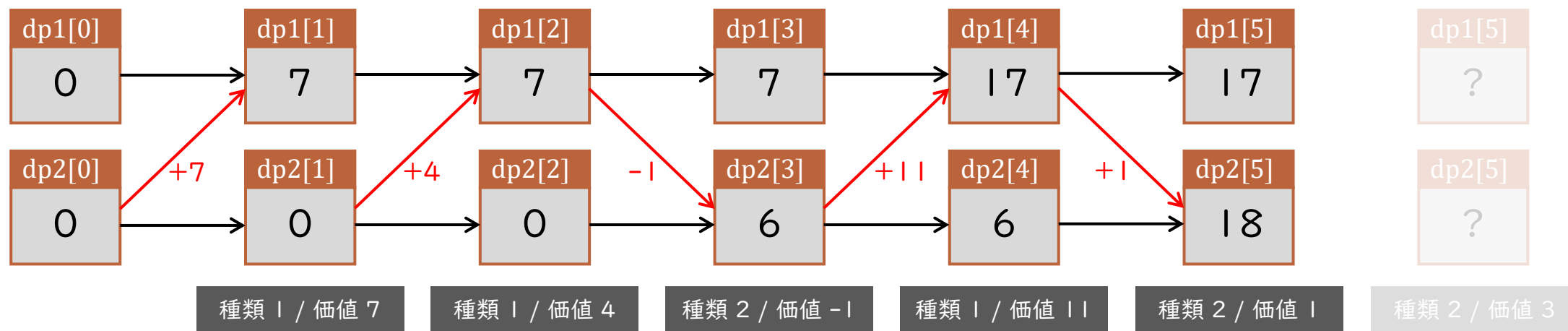
5

小課題 5 の解法

41 / 49

- 難しいので、たとえば以下のようなケースを考えよう

- $dp1[5] = dp1[4] = 17$ [最後に残した絵が種類 1 のときの合計価値最大]
- $dp2[5] = \max(dp2[4], dp1[4] + 1) = 18$ [最後に残した絵が種類 2 のときの合計価値最大]



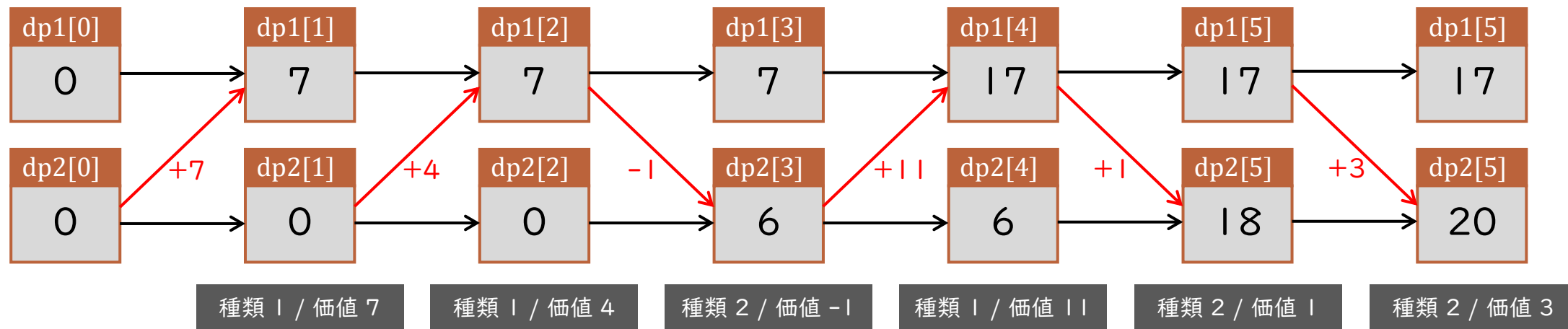
5

小課題 5 の解法

42 / 49

- 難しいので、たとえば以下のようなケースを考えよう

- $dp1[6] = dp1[5] = 17$ [最後に残した絵が種類 1 のときの合計価値最大]
- $dp2[6] = \max(dp2[5], dp1[5] + 3) = 20$ [最後に残した絵が種類 2 のときの合計価値最大]



5

小課題 5 の解法

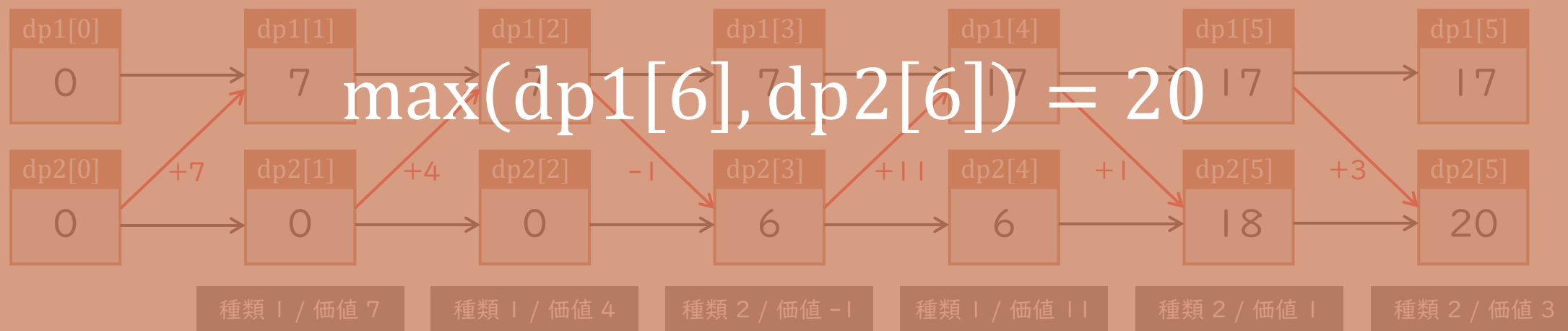
43 / 49

- 難しいので、たとえば以下のようなケースを考えよう

- $dp1[6] = dp1[5] = 17$ [最後に残した絵が種類 1 のときの合計価値最大]

- $dp2[6] = \max(dp2[5], dp1[5] + 3) = 20$ [最後に残した絵が種類 2 のときの合計価値最大]

求める答え（残す絵の合計価値の最大）は



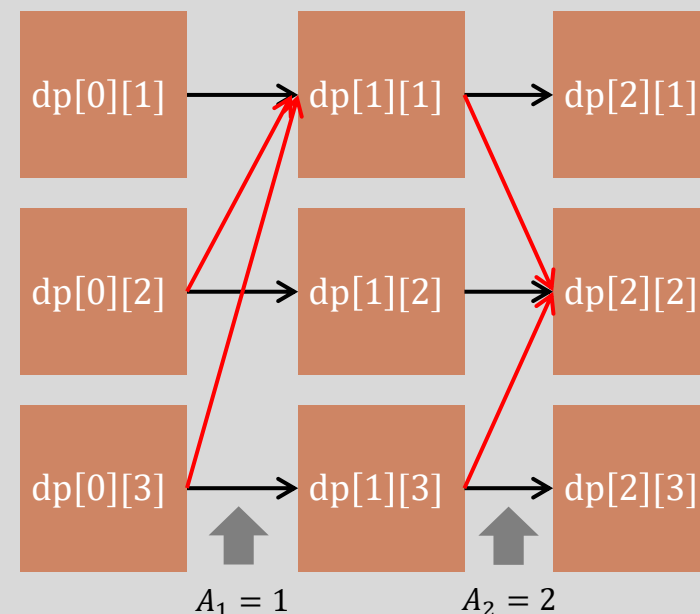
5

小課題 5 の解法

44 / 49

M が一般の場合も、同じようにして解ける（詳しい方針は以下の通り）

- $dp[pos][col]$ を「絵 $1, 2, \dots, pos$ を考えたときに、最後に残した絵の種類が col である場合の合計価値最大」とする
- 最終的な答えは $dp[pos][1], \dots, dp[pos][M]$ の最大値となる
- イメージ図は右の通り



5

小課題 5 の実装

45 / 49

```
// C++ での実装
// 動的計画法
for (int i = 1; i <= M; i++) dp[0][i] = 0;
for (int i = 1; i <= N; i++) {
    for (int j = 1; j <= M; j++) dp[i][j] = dp[i-1][j];
    for (int j = 1; j <= M; j++) {
        if (j == A[i]) continue;
        dp[i][A[i]] = max(dp[i][A[i]], dp[i-1][j] + V[i]);
    }
}

// 答えを出力
int Answer = 0;
for (int i = 1; i <= M; i++) Answer = max(Answer, dp[N][i]);
cout << Answer << endl;
```

- このアルゴリズムを実装すると、左のようになる
- 計算量は $O(NM)$ なので、 $M \leq 100$ ならば通る

88点

小課題 6

追加の制約はない

6

小課題 6 の解法

47 / 49

- 絵 i を置いた場合、 $dp[i][A_i]$ の値は以下のうち最大の値

- $dp[i-1][1] + V_i$

⋮

- $dp[i-1][A_i-1] + V_i$

- $dp[i-1][A_i+1] + V_i$

⋮

- $dp[i-1][M] + V_i$

そこで最大の値となり得るのは

$dp[i-1][1], \dots, dp[i-1][N]$ のうち上位 2 つだけ

注: 1 位が $dp[i-1][A_i]$ である場合に限り、左の最大の値が 2 位のものになる

6

小課題 6 の解法

48 / 49

- したがって、 $dp[i][1], \dots, dp[i][M]$ 全部をメモする必要はない
- メモする必要があるのは 上位 2 つだけ
 - $dp1[i]$: $dp[i][1], \dots, dp[i][M]$ の中で最も大きい値
 - $dp2[i]$: $dp[i][1], \dots, dp[i][M]$ の中で 2 番目に大きい値
- そうすると、考えるべき状態の数が $2N$ 通りに減る
- 計算量は $O(N)$ [実装はやや大変です]

100点

6

小課題 6 の実装例

49 / 49

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int N, M;
int A[150009], V[150009];
pair<int, int> dp1[150009];
pair<int, int> dp2[150009];

int main() {
    // Step #1. 入力
    cin >> N >> M;
    for (int i = 1; i <= N; i++) {
        cin >> A[i] >> V[i];
    }

    // Step #2. 配列の初期化
    for (int i = 1; i <= N; i++) {
        dp1[i] = make_pair(-2000000000, -1);
        dp2[i] = make_pair(-2000000000, -1);
    }
    dp1[1] = make_pair(V[1], A[1]);
    dp2[1] = make_pair(0, -1);
```

```
// Step #3. 動的計画法
for (int i = 2; i <= N; i++) {
    vector<pair<int, int>> vec;
    if (dp1[i - 1].second != A[i]) vec.push_back(make_pair(dp1[i - 1].first + V[i], A[i]));
    if (dp2[i - 1].second != A[i]) vec.push_back(make_pair(dp2[i - 1].first + V[i], A[i]));
    vec.push_back(make_pair(dp1[i - 1].first, dp1[i - 1].second));
    vec.push_back(make_pair(dp2[i - 1].first, dp2[i - 1].second));
    sort(vec.begin(), vec.end());
    reverse(vec.begin(), vec.end());

    dp1[i] = vec[0];
    for (int j = 1; j < (int)vec.size(); j++) {
        if (vec[j].second == vec[0].second) continue;
        dp2[i] = vec[j];
        break;
    }
}

// Step #4. Output
cout << dp1[N].first << endl;
return 0;
}
```

問題 6 「タクシー 2」 解説

日本情報オリンピック 第2回女性部門 本選

解説：米田 寛峻 (square1001)

問題の説明

2 / 33

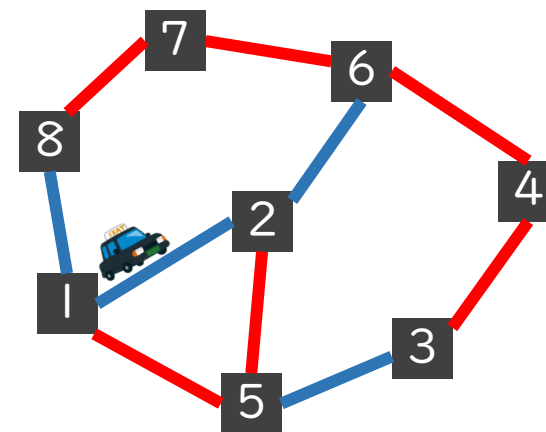
N 個の町と M 本の道（赤色 or 青色）があり、その情報が与えられます

- 赤い道を通ると、所持金が x 円 $\rightarrow x - 1$ 円になります
- 青い道を通ると、所持金が x 円 $\rightarrow \lfloor x \div 2 \rfloor$ 円になります

「町 1 から出発して、町 T まで 1 円以上残して到着するためには、最初に何円の所持金を持っておく必要があるか？」

という形式の質問に Q 回答えてください

ただし L 円でも足りない場合は “Large” と答えること



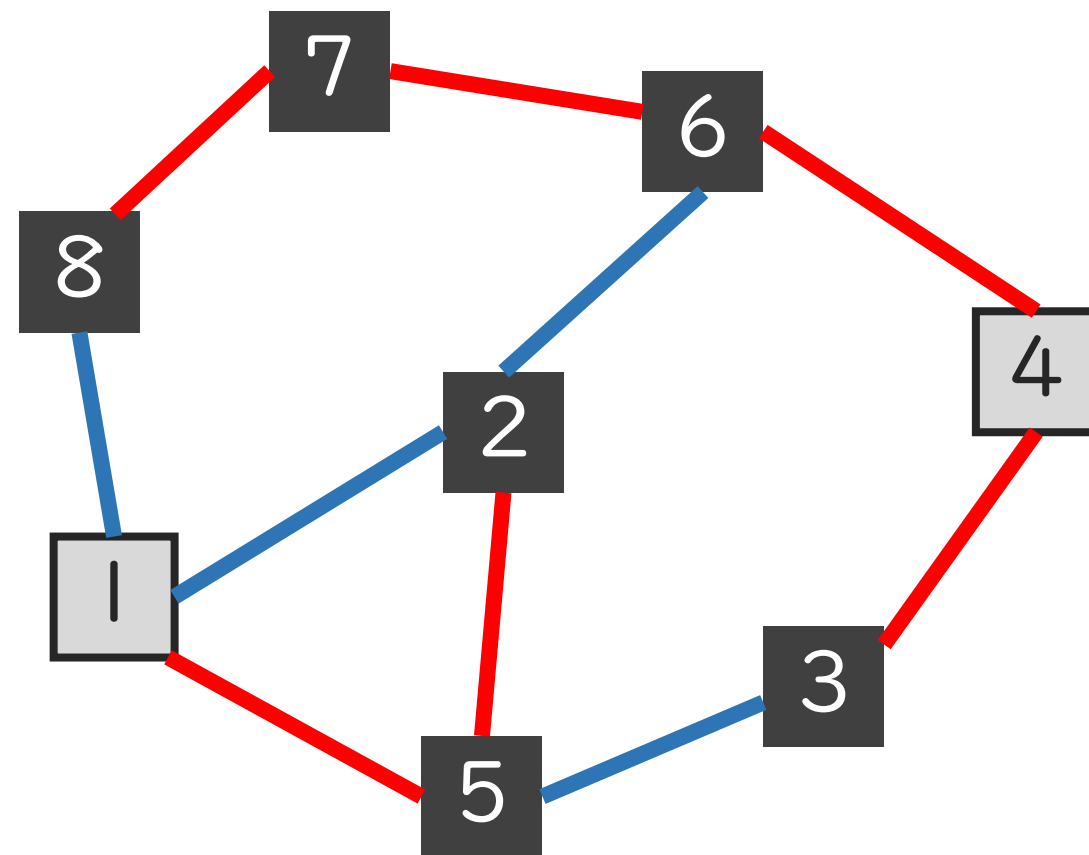
具体的な例

3 / 33

右図の例で、町 1 から町 4 に行きたい場合

町 1 $\rightarrow$ 5 $\rightarrow$ 3 $\rightarrow$ 4 の順に行くのが最適

赤色・青色・赤色の道を順に通るので、最初の所持金が 5 円なら OK (お金が 5 $\rightarrow$ 4 $\rightarrow$ 2 $\rightarrow$ 1 円と減っていく)



各小課題の制約

4 / 33

この問題には 7 つの小課題があり、できるだけ性能がよいプログラムを書くほど高得点が得られるようになっています（例えば「道路が一直線の場合」を解くと $9 + 19 + 19 = 47$ 点）

小課題（得点）	道路は一直線か？	$Q = 1$ か？	N, M の制約	その他の制約
#1 (9 点)	✓	✓	100	$L \leq 100$
#2 (19 点)	✓	✓	200,000	-
#3 (19 点)	✓		200,000	-
#4 (16 点)		✓	50,000	道はすべて赤色
#5 (20 点)		✓	50,000	-
#6 (12 点)			50,000	-
#7 (5 点)			200,000	-

小課題 1 (9 点)

道路網は一直線 $\cdot N \leq 100 \cdot Q = 1 \cdot L \leq 100$

小課題 1 の解法

6 / 33

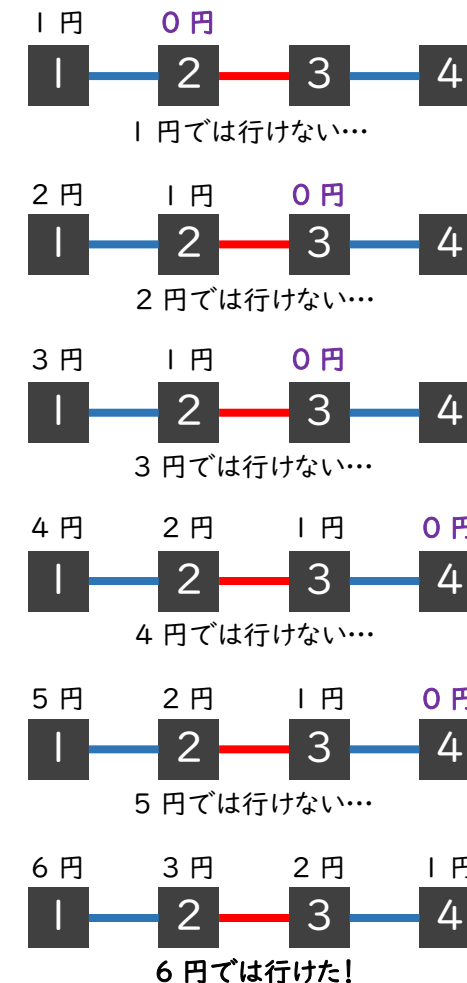
小課題 1~3 では、道が一直線になっています

この場合、タクシーは「町 1 → 2 → ... → T_i 」の順に通ることになるので、所持金が決まれば簡単にシミュレーションできます

以下のアルゴリズムで解くと、小課題 1 に正解できます

アルゴリズム

- 「1 円でいけるかな?」「2 円でいけるかな?」...「L 円で行けるかな?」と試していったときに、初めて行けたときの所持金が答え
- L 円でも無理だったら、答えは "Large" になる
- 所持金を決めたときのシミュレーションは計算量 $O(N)$ にかかるので、全体計算量 $O(NL)$ で解ける



→ 答えは「6」

小課題 1 のプログラム例

7 / 33

C++ と Python での実装例です（入力・出力の部分は省略しています）

C++

```
int answer = -1; // answer = -1 の場合 "Large" とする
for (int money = 1; money <= L; money++) {
    int x = money;
    for (int i = 1; i < T; i++) {
        if (C[i] == 1) x -= 1;
        if (C[i] == 2) x /= 2;
    }
    if (x >= 1) {
        answer = money;
        break;
    }
}
```

Python

```
answer = -1 # answer = -1 の場合 "Large" とする
for money in range(1, L + 1):
    x = money
    for i in range(1, T):
        if C[i] == 1:
            x -= 1
        if C[i] == 2:
            x //= 2
    if x >= 1:
        answer = money
        break
```



ここまでで 9 点

小課題 2 (28 点)

道路網は一直線 $\cdot N \leq 200\,000 \cdot Q = 1$

小課題 2 の解法：二分探索

9 / 33

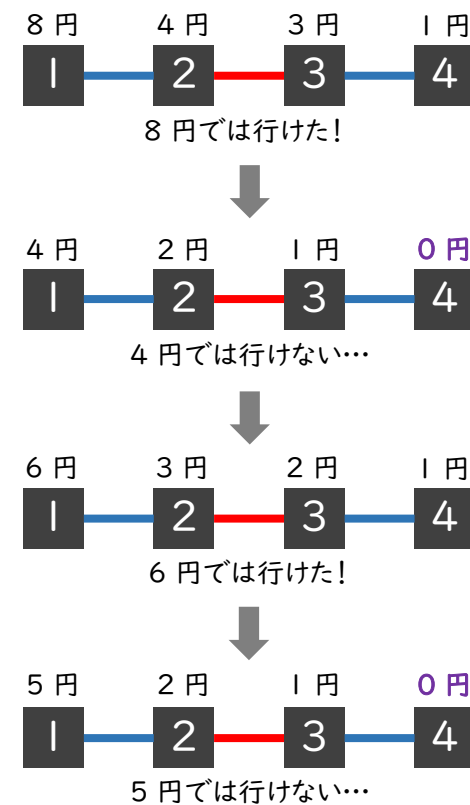
小課題 1 では「1 円で行けるか?」「2 円で行けるか?」… と順々に確かめていったが、このようにするのは非効率的です

ここで、答えを「二分探索」してみましょう

答えを「二分探索」とは?

- 例えば、答えが 1~16 であることが分かっていたとする
- 最初の所持金を 8 円としてシミュレーションを行うと…?
 - 行けた場合、答えが 1~8 と分かる
 - 行けなかった場合、答えが 9~16 と分かる
- このようなことを繰り返すと、答えの範囲が 16 個 → 8 個 → 4 個 → 2 個 → 1 個と半分ずつに減っていき、効率的に答えが探せる

具体例: $L = 15$ の場合



小課題 2 の解法：二分探索

10 / 33

二分探索では $\log_2 L$ 回ほどシミュレーションするので、計算量 $O(N \log L)$ で解けました

C++ と Python における二分探索のイメージを載せておきます

※ ここでは、`check(x)` を「最初の所持金 x 円で行けるか」を返す関数とします

C++

```
int l = 1, r = L + 1;
while (r - l >= 1) {
    int m = (l + r) / 2;
    if (check(m) == true) r = m;
    else l = m + 1;
}
if (r == L + 1) {
    cout << "Large" << endl;
}
else {
    cout << r << endl;
}
```

Python

```
l, r = 1, L + 1
while r - l >= 1:
    m = (l + r) // 2
    if check(m) == True:
        r = m
    else:
        l = m + 1
if r == L + 1:
    print("Large")
else:
    print(r)
```



ここまでで 28 点

小課題 2 の解法：後ろから考える

11 / 33

小課題 2 は「後ろから考える」方法で解くこともできます



例えば右図の例で、町 1 から町 6 に行くとき…

- 町 6 で 1 円残すためには、町 5 では最低 $1 + 1 = 2$ 円はないといけない
- 町 5 で 2 円残すためには、町 4 では最低 $2 + 1 = 3$ 円はないといけない
- 町 4 で 3 円残すためには、町 3 では最低 $3 \times 2 = 6$ 円はないといけない
- 町 3 で 6 円残すためには、町 2 では最低 $6 + 1 = 7$ 円はないといけない
- 町 2 で 7 円残すためには、町 1 では最低 $7 \times 2 = 14$ 円はないといけない

このようにして答えが「14 円」と分かります



小課題 2 の解法：後ろから考える

12 / 33

したがって、1 つの質問あたり計算量 $O(Q)$ で解くことができます

C++ と Python での実装例です（入力の部分は省略しています）

C++

```
int answer = 1;
for (int i = T - 1; i >= 1; i--) {
    if (C[i] == 1) answer += 1;
    if (C[i] == 2) answer *= 2;
    if (answer > L) break;
}
if (r > L) {
    cout << "Large" << endl;
}
else {
    cout << r << endl;
}
```

Python

```
answer = 1
for i in range(T - 1, 0, -1):
    if C[i] == 1:
        answer += 1
    if C[i] == 2:
        answer *= 2
    if answer > L:
        break
if r == L + 1:
    print("Large")
else:
    print(r)
```



ここまでで 28 点

小課題 3 (47 点)

道路網は一直線 $\cdot N \leq 200\,000 \cdot Q \leq 200\,000$

小課題 3 の解法

14 / 33

小課題 3 ではたくさんの質問が与えられます

すべての町に対して、答え = 「必要な所持金の最小値」を求めることになります

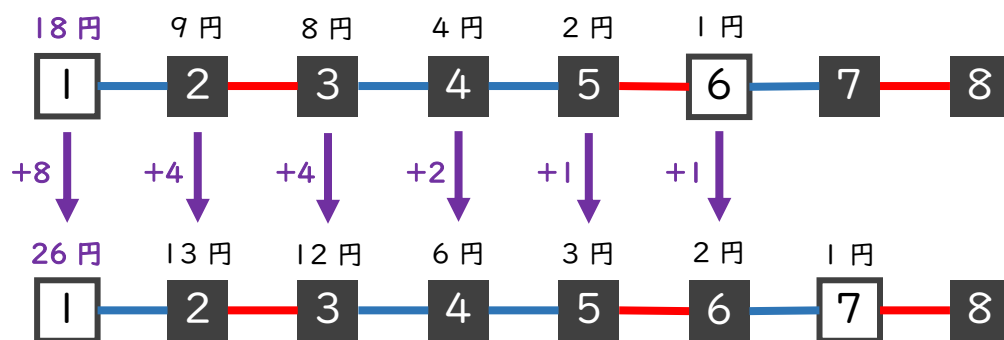


小課題 3 の解法

15 / 33

実は、 $T = 2, 3, 4, \dots, N$ のときの答えを順々に計算していくと効率的です

例えば、先ほどの例で $T = 6$ から $T = 7$ に変化したとき、答えがどう変わるか注目しましょう



駅 6 と駅 i までの青い道が k_i 個のとき
駅 i 時点での所持金が 2^{k_i} 円上がっている!

このように、 $T = j$ から $T = j + 1$ に変化したとき、答えは $2^{(\text{駅 1 と駅 } j \text{ の間の青い道の個数})}$ 円だけ上がります

こうして、すべての T に対する答えが、計算量 $O(N)$ で求められます

小課題 3 の解法

16 / 33

C++ と Python での実装例です（入力の部分は省略しています）

C++

```
ans[1] = 1;
for (int i = 2; i <= N; i++) {
    ans[i] = L + 1;
}
int blues = 0;
for (int i = 1; i < N; i++) {
    // (1 << blues) は 2 の blues 乗
    ans[i + 1] = ans[i] + (1 << blues);
    if (ans[i + 1] > L) break;
    if (C[i] == 2) blues++;
}
for (int i = 1; i <= Q; i++) {
    if (ans[T[i]] > L) {
        cout << "Large" << endl;
    }
    else {
        cout << ans[T[i]] << endl;
    }
}
```

Python

```
ans = [ L + 1 ] * (N + 1)
ans[1] = 1
blues = 0
for i in range(1, N):
    ans[i + 1] = ans[i] + (2 ** blues)
    if ans[i + 1] > L:
        break
    if C[i] == 2:
        blues += 1
for i in range(Q):
    if ans[T[i]] > L:
        print("Large")
    else:
        print(ans[T[i]])
```



ここまでで 47 点

小課題 4 (12 点)

$N, M \leq 50\,000 \cdot Q = 1$ ・ タクシーはすべて赤色

グラフの問題として考えよう

18 / 33

小課題 4 以降は、グラフの問題として解くことになります

グラフとは？

モノとモノの結びつき方を表すネットワーク構造を **グラフ** といいます

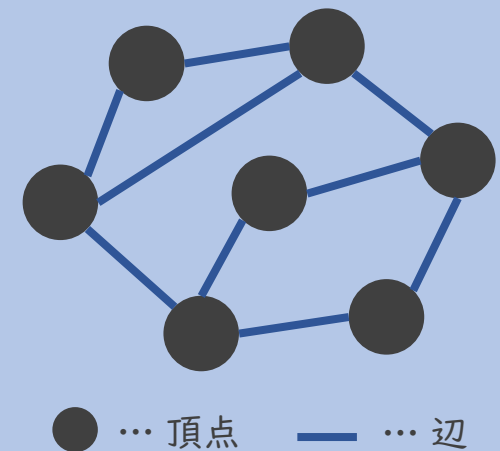
グラフは「頂点」と「辺」からなります

道路網もグラフで表すことができ、町が「頂点」、道が「辺」に対応

詳しく知りたい方はこちら

- ・ 「アルゴリズム×数学」 p.162～168
- ・ <https://qiita.com/square1001/items/6d414167ca95c97bd8b2> 3 節

※ 棒グラフや折れ線グラフなどの「グラフ」とは全く別の概念であることに注意してください



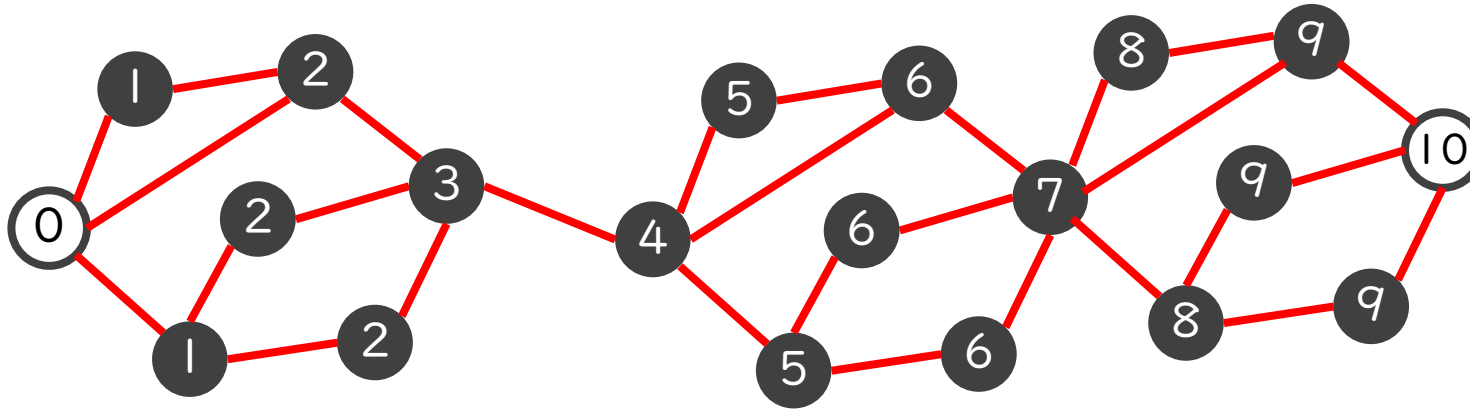
小課題 4 の解法

19 / 33

小課題 4 の制約は「 $C_i = 1$ 」すなわちすべての道が赤色です

つまり、頂点 1 から頂点 T まで、最も少ない辺の本数たどって移動する方法を求めたいです

- なぜなら、(必要な所持金) = (通った辺の本数) + 1 だから



(丸の中の数はスタート地点からの最短経路の長さを表す)

小課題 4 の解法

20 / 33

実は、(重みなし) グラフの最短経路は「幅優先探索 (BFS)」というアルゴリズムを使うと計算量 $O(N + M)$ で求められます

幅優先探索 (BFS) について知りたい方はこちらを読んでください

- 「アルゴリズム×数学」p.173~177
- <https://qiita.com/drken/items/996d80bcae64649a6580>

よって、小課題 4 は計算量 $O(N + M)$ で解けました



ここまでで 63 点

小課題 5 (64 点)

$$N, M \leq 50\,000 \cdot Q = 1$$

小課題 2 を思い出そう

22 / 33

小課題 5 では、赤い辺も青い辺もある場合を解かなければなりません

ここで、小課題 2 の解法を思い出してみましょう

小課題 2: 解法 1

答えを二分探索

小課題 2: 解法 2

後ろから考える

実は、どちらのアイデアも、小課題 5（一般のグラフの場合）の解法につながっています

本解説では、「後ろから考える」アイデアのほうを説明します

※「答えを二分探索」のアイデアを小課題 5 に適用すると計算量が $O(M \log M \log L)$ となるため、実装によっては TLE（実行時間制限オーバー）となる可能性があります。

小課題 5 の解法

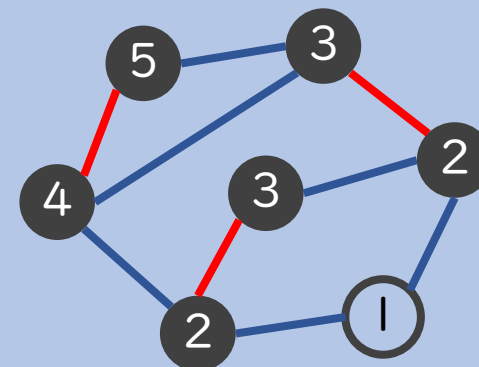
23 / 33

後ろから考えた問題

頂点 T からスタートして、頂点 1 でゴールすることを考えます

- 赤い辺を通ると、所持金が +1 されます
- 青い辺を通ると、所持金が $\times 2$ されます

最初 1 円からスタートするとき、ゴール時点での最小の所持金はいくらでしょうか？



(丸の中の数は、スタート地点から各頂点に到達する時点での最小の所持金)

$\text{dist}[v]$ = (頂点 v にいる際の最小の所持金) とすると、 $\text{dist}[1]$ がこの問題の答えです

これは、どうやって求めるのでしょうか？

小課題 5 の解法

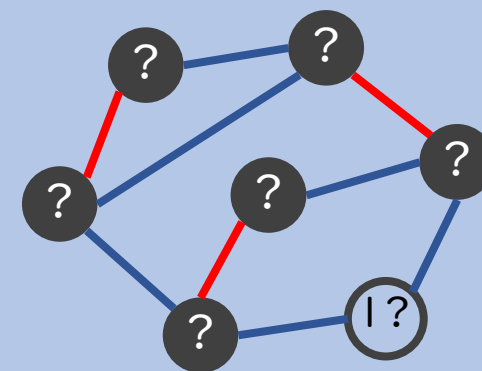
24 / 33

実は、これはダイクストラ法（重み付きグラフの最短経路問題を解くアルゴリズム）と同様のアルゴリズムで求めることができます

アルゴリズム

各頂点は「距離が未確定の頂点」または「距離が確定した頂点」のいずれかであり、最初はすべて未確定とする

1. 最初 $\text{dist}[T] = 1$ とし、それ以外の頂点の距離の暫定値を $\text{dist}[i] = \infty$ とする（すべて未確定）
2. 以下の処理を N 回繰り返す
 - 未確定の頂点のうち $\text{dist}[v]$ が最小の頂点 v を選び、距離を確定させる
 - 頂点 v と隣接する頂点の $\text{dist}[i]$ よりも、 $\text{dist}[v] + 1$ （赤辺の場合）あるいは $\text{dist}[v] \times 2$ （青辺の場合）の方が小さければ、 $\text{dist}[i]$ を更新する
3. 結果として、 $\text{dist}[1], \text{dist}[2], \dots, \text{dist}[N]$ がすべて求まった（答えは $\text{dist}[1]$ ）

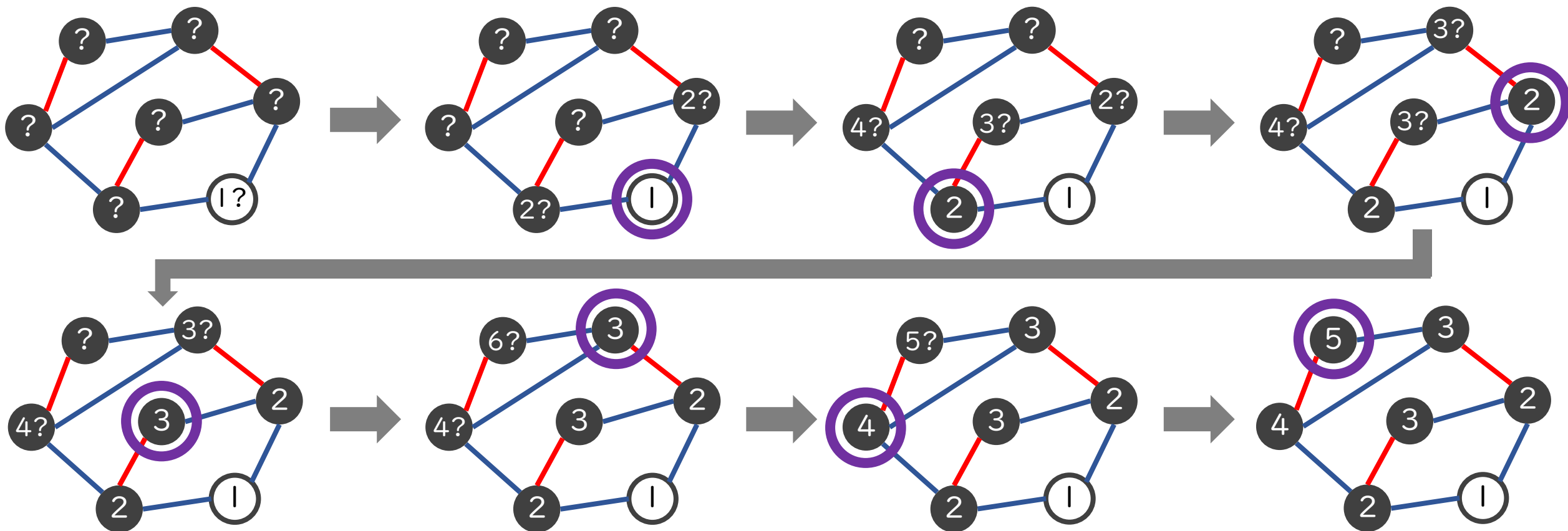


スタート時点での状態

小課題 5 の解法

25 / 33

前ページの例では、アルゴリズムは以下のように動作します



小課題 5 の解法

26 / 33

未確定の頂点のなかで $\text{dist}[v]$ が最小のものを取り出すのに「優先度付きキュー」を使うと全体計算量 $O(M \log M)$ でこの問題が解けます

説明が難しかったかもしれませんが、ダイクストラ法とほぼ同じアルゴリズムです

ダイクストラ法について知りたい方はこちら

- 「問題解決力を鍛える! アルゴリズムとデータ構造」p. 254~264
- <https://www.youtube.com/watch?v=e6X2gDTZYCQ> (早大・早水桃子先生の動画)



ここまでで 83 点

満点解法 (100 点)

$$N, M, Q \leq 200\,000$$

小課題 3 を思い出そう

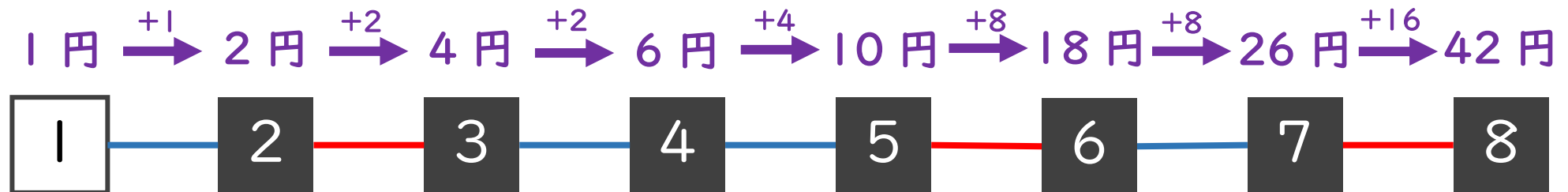
28 / 33

小課題 6・7 ではたくさんの質問が与えられます

$T = 2, 3, \dots, N$ すべてに対し、答え = 「必要な所持金の最小値」を求めることになります

小課題 3 の解法を思い出してみましょう

前から順に答えを計算していき、辺を通るごとに必要な所持金が $2^{(\text{通った青い道の個数})}$ 円増えます



つまり「通った青い辺の個数」によって、次の頂点に移動するコストが決まります

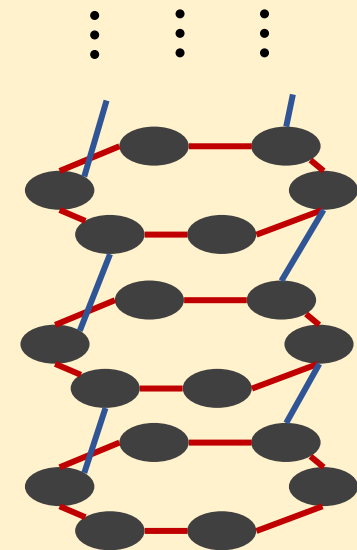
$\text{dist}[\text{level}][\text{pos}] = (\text{青い辺を } \text{level} \text{ 個通って頂点 } \text{pos} \text{ に行くときの、最初の所持金の最小値})$
とすると、以下のように最短経路問題に帰着できます

最短経路問題にする

$\text{dist}[\text{level}][\text{pos}]$ の状態を「状態 $(\text{level}, \text{pos})$ 」ということにする

- 頂点 pos と頂点 i を赤い辺がつないでいるとき、状態 $(\text{level}, \text{pos})$
→ 状態 (level, i) にコスト 2^{level} で行ける
- 頂点 pos と頂点 i を青い辺がつないでいるとき、状態 $(\text{level}, \text{pos})$
→ 状態 $(\text{level} + 1, i)$ にコスト 2^{level} で行ける

最終的な答えは $\text{dist}[0][T], \text{dist}[1][T], \text{dist}[2][T], \dots$ の最小値



満点解法 A

30 / 33

N 頂点 M 辺の重み付きグラフの最短経路は、ダイクストラ法を使って計算量 $O(M \log M)$ で求められます ※ 重み付きグラフとは、辺ごとにコスト（重み）が異なるグラフのことです

ダイクストラ法を行うと、スタートの頂点 s が決まっているときの頂点 i までの最短距離 $dist[i]$ が、 $i = 1, 2, \dots, N$ すべてに対して求まります

この問題では…？

$level = 0, 1, 2, \dots, K - 1$ に対して最短経路を求める場合、 NK 頂点 MK 辺程度のグラフの最短経路を求めるから、計算量 $O(MK \log MK)$ かかります

青い辺は最大で M 個あるから、 $K = M$ で計算量 $O(M^2 \log M)$ …？

満点解法 A

31 / 33

実は「青い辺を 29 回までしか通らない」を仮定してもよいです!

この理由は、もし青い辺を 30 回以上通れば、距離が L ($\leq 10^9$) を超えてしまうからです
このとき結果が “Large” になるから、計算しても無駄です

$K = \lceil \log_2 L \rceil$ (あるいは $K = 30$) にしてよいので、最短経路問題を計算量 $O(M \log M \log L)$ で解くことにより、この問題のすべての T に対する答えが求まります

※ 上手く実装すれば、C++ や Java などの言語で実行時間制限 4 秒に間に合い、満点を得ることができます。多少遅い実装をしても、95 点が得られます。



ここまでで 100 点

この問題は、計算量 $O(M \log L)$ で解くこともできます

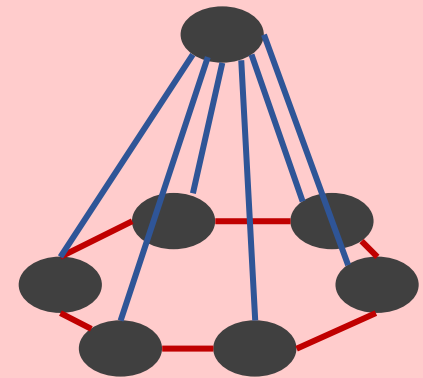
$level$ が同じ N 頂点の「層」だけを考えると、すべての辺のコストは 2^{level} で等しいです

したがって、各層ごとに $dist[level][1], dist[level][2], \dots, dist[level][N]$ を求めると、ダイクストラ法ではなく幅優先探索 (BFS) のように解くことができます

解きたい最短経路問題

N 頂点 M 辺で、辺のコストがすべて C のグラフを考える

- このグラフに頂点 $N + 1$ を追加し、各 i に対して頂点 i と頂点 $N + 1$ をコスト w_i の辺でつないだ新しいグラフを作る
- このグラフに対して、頂点 $N + 1$ から各頂点までの最短距離を求めたい



満点解法 B

33 / 33

実は、ダイクストラ法を 2 つのキュー Q_1, Q_2 を使って再現できます

- Q_1 : 頂点 $N + 1$ からコスト w_i の辺を通ったときに push されるキュー
- Q_2 : 頂点 $N + 1$ 以外から辺を通ったときに push されるキュー

Q_1 と Q_2 が最短距離順に並べられているならば、各ステップでは Q_1 の先頭と Q_2 の先頭のうち $\text{dist}[i]$ が小さい方を取り出せばよいから、 $w_1, w_2, \dots, w_N$ のソート順が分かっている状態であれば計算量 $O(N + M)$ で前述の最短経路問題が解けます

したがって、以下の手順で、全体計算量 $O(M \log L)$ で解けます

1. まず $\text{level} = 0$ の層の最短経路を BFS で求める。 $\text{dist}[0][1], \text{dist}[0][2], \dots, \text{dist}[0][N]$ のソート順は、キューが pop された順番で分かる。
2. 次に $\text{level} = 1$ の層の最短経路を前述の方法で求める。 $\text{dist}[1][1], \text{dist}[1][2], \dots, \text{dist}[1][N]$ のソート順は、キューが pop された順番で分かる。
3. (中略)
4. 最後に $\text{level} = 29$ の層の最短経路を前述の方法で求める。

この方法を使うと、特に C++ では実行時間制限の 10 分の 1 未満しか使わずに正解することもできます